Java Programming Lab

50 Practice Tasks: Basic to Critical Thinking

Name: Anaya Azam Roll No: 2026-CYS-S-12 Date: 12th October,2026

Instructions

- Write a complete Java program for each task. Keep the code short and clean.
- Tasks marked [OOP] must use classes and objects. Tasks marked [Non-OOP] should be solved with methods and logic only.
- Do not use ready-made library shortcuts where a task says not to.
- Test every program with normal input and with edge cases (zero, negative, empty, duplicates).

Overview

Level	Tasks	OOP	Non-OOP
Level 1: Basic	1 to 12	2	10
Level 2: Intermediate	13 to 30	6	12
Level 3: Advanced	31 to 42	5	7
Level 4: Expert (Critical Thinking)	43 to 50	4	4

Level 1: Basic

Task 1 [Non-OOP]

Read an integer and print whether it is even or odd without using the % operator.

Code:

```
static void task01() {
    int n = readInt("Enter an integer: ");
    if (n / 2 * 2 == n) {
        System.out.println(n + " is even");
    } else {
        System.out.println(n + " is odd");
    }
}
```

Output:

```
Enter an integer: 12
12 is even
```

```
Enter an integer: 0
0 is even
```

```
Enter an integer: -15
-15 is odd
```

Task 2 [Non-OOP]

Swap two int variables without using a third variable. Print the values before and after swapping.

Code:

```
static void task02() {
    int a = readInt("Enter a: ");
    int b = readInt("Enter b: ");
    System.out.println("Before: a = " + a + ", b = " + b);

    a = a + b;
    b = a - b;
    a = a - b;
    System.out.println("After:  a = " + a + ", b = " + b);
}
```

Output:

```
Enter a: 34
Enter b: 96
Before: a = 34, b = 96
After:  a = 96, b = 34
```

Task 3 [Non-OOP]

Read three numbers and print the middle (median) one without using arrays or sorting.

Code:

```
static void task03() {
    double a = readDouble("Enter first number: ");
    double b = readDouble("Enter second number: ");
    double c = readDouble("Enter third number: ");

    double median;
    if ((a >= b && a <= c) || (a <= b && a >= c)) {
        median = a;
    } else if ((b >= a && b <= c) || (b <= a && b >= c)) {
        median = b;
    } else {
        median = c;
    }
    System.out.println("Median = " + median);
}
```

Output:

```
Enter first number: 46
Enter second number: 37
Enter third number: 75
Median = 46.0
```

Task 4 [OOP]

Create a class Rectangle with width and height, a constructor, and methods area(), perimeter() and isSquare(). Create two objects and print the one with the larger area.

Code:

```
static class Task04 {
    static class Rectangle {
        private double width;
        private double height;

        Rectangle(double width, double height) {
            this.width = width;
            this.height = height;
        }
        double area() { return width * height; }
        double perimeter() { return 2 * (width + height); }
        boolean isSquare() { return width == height; }

        public String toString() { return "Rectangle " + width + " x " + height; }
    }
    static void run() {
        Rectangle r1 = new Rectangle(5, 4);
        Rectangle r2 = new Rectangle(6, 6);
```

```
        System.out.println(r1 + ": area=" + r1.area() + ", perimeter=" + r1.perimeter()
                + ", square? " + r1.isSquare());
        System.out.println(r2 + ": area=" + r2.area() + ", perimeter=" + r2.perimeter()
                + ", square? " + r2.isSquare());

        if (r1.area() > r2.area()) {
            System.out.println("Larger area: " + r1);
        } else if (r2.area() > r1.area()) {
            System.out.println("Larger area: " + r2);
        } else {
            System.out.println("Both have the same area");
        }
    }
}
```

Output:

```
Rectangle 5.0 x 4.0: area=20.0, perimeter=18.0, square? false
Rectangle 6.0 x 6.0: area=36.0, perimeter=24.0, square? true
Larger area: Rectangle 6.0 x 6.0
```

Task 5 [Non-OOP]

Print a number triangle where row i contains the digit i repeated i times (1, 22, 333, ...) for a user-given n.

Code:

```
static void task05() {
    int n = readInt("Enter n: ");
    for (int row = 1; row <= n; row++) {
        for (int count = 1; count <= row; count++) {
            System.out.print(row);          // print the row number "row" times
        }
        System.out.println();
    }
}
```

Output:

```
Enter n: 10
1
22
333
4444
55555
666666
7777777
88888888
999999999
10101010101010101010
```

Task 6 [Non-OOP]

Read an integer and print its digits in reverse order without converting it to a String. Trailing zeros must be kept (1200 prints 0021).

Code:

```
static void task06() {
    long n = readInt("Enter an integer: ");
    if (n < 0) {
        System.out.print("-");
        n = -n;
    }
    if (n == 0) {
        System.out.print(0);
    }
    while (n > 0) {
        System.out.print(n % 10);
        n = n / 10;
    }
    System.out.println();
}
```

Output:

```
Enter an integer: 234567
765432
```

```
Enter an integer: 1200
0021
```

Task 7 [Non-OOP]

Repeatedly add the digits of a number until a single digit remains (for example 9875 gives 29, then 11, then 2).

Code:

```
static int sumOfDigits(int n) {
    int sum = 0;
    while (n > 0) {
        sum = sum + n % 10;
        n = n / 10;
    }
    return sum;
}
static void task07() {
    int n = readInt("Enter a number: ");
    if (n < 0) n = -n;
    System.out.print(n);
    while (n >= 10) {
        n = sumOfDigits(n);
        System.out.print(" -> " + n);
    }
    System.out.println();
}
```

Output:

```
Enter a number: 236585
236585 -> 29 -> 11 -> 2
```

```
Enter a number: 123456
123456 -> 21 -> 3
```

```
Enter a number: -37595
37595 -> 29 -> 11 -> 2
```

Task 8 [OOP]

Create a class Counter that tracks how many objects have been created (static) and how many times each object has been incremented (instance). Create 3 objects, increment them different numbers of times, and print both kinds of counts.

Code:

```
static class Task08 {
    static class Counter {
        private static int objectsCreated = 0;
        private int timesIncremented = 0;
        Counter() { objectsCreated++; }
        void increment() { timesIncremented++; }
        int getTimesIncremented() { return timesIncremented; }
        static int getObjectsCreated() { return objectsCreated; }
    }
    static void run() {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();
        c1.increment(); c1.increment();
        for (int i = 0; i < 5; i++) c2.increment();
        c3.increment();
        System.out.println("Objects created (static): " + Counter.getObjectsCreated());
        System.out.println("c1 incremented: " + c1.getTimesIncremented());
        System.out.println("c2 incremented: " + c2.getTimesIncremented());
        System.out.println("c3 incremented: " + c3.getTimesIncremented());
    }
}
```

Output:

```
Objects created (static): 3
c1 incremented: 2
c2 incremented: 5
c3 incremented: 1
```

Task 9 [Non-OOP]

Write a method isPrime(int n) that loops only up to the square root of n. Use it to print all primes from 1 to 100.

Code:

```
static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
static void task09() {
    System.out.println("Primes from 1 to 100:");
    for (int i = 1; i <= 100; i++) {
        if (isPrime(i)) System.out.print(i + " ");
    }
    System.out.println();
}
```

Output:

```
Primes from 1 to 100:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

Task 10 [Non-OOP]

Read a String and count vowels, consonants, digits and other characters using a single loop.

Code:

```
static void task10() {
    String text = readText("Enter a string: ");
    int vowels = 0, consonants = 0, digits = 0, others = 0;

    for (int i = 0; i < text.length(); i++) {
        char ch = Character.toLowerCase(text.charAt(i));
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u') {
            vowels++;
        } else if (ch >= 'a' && ch <= 'z') {
            consonants++;
        } else if (ch >= '0' && ch <= '9') {
            digits++;
        } else {
            others++;
        }
    }
    System.out.println("Vowels: " + vowels + ", Consonants: " + consonants
            + ", Digits: " + digits + ", Others: " + others);
}
```

Output:

```
Enter a string: Read a String and count vowels, consonants, digits and other characters using a single loop.
Vowels: 27, Consonants: 48, Digits: 0, Others: 17
```

Task 11 [Non-OOP]

Print the first N Fibonacci numbers (N from the user) using only three variables, no arrays.

Code:

```
static void task11() {
    int n = readInt("Enter N: ");
    long first = 0, second = 1, next;
    for (int i = 0; i < n; i++) {
        System.out.print(first + " ");
        next = first + second;
        first = second;
        second = next;
    }
    System.out.println();
}
```

Output:

```
Enter N: 15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Task 12 [Non-OOP]

Check whether a String is a palindrome, ignoring case, spaces and punctuation ("A man, a plan, a canal: Panama" must return true).

Code:

```
static boolean isPalindrome(String text) {
    String clean = "";
    for (int i = 0; i < text.length(); i++) {
        char ch = text.charAt(i);
        if (Character.isLetterOrDigit(ch)) {
            clean = clean + Character.toLowerCase(ch);
        }
    }
    int left = 0;
    int right = clean.length() - 1;
    while (left < right) {
        if (clean.charAt(left) != clean.charAt(right)) return false;
        left++;
        right--;
    }
    return true;
}
static void task12() {
    String text = readText("Enter a string: ");
    System.out.println(isPalindrome(text));
}
```

Output:

```
Enter a string: A man, a plan, a canal: Panama
true
```

```
Enter a string: hello there
false
```

Level 2: Intermediate

Task 13 [OOP]

Create a BankAccount class with a private balance and methods deposit(), withdraw() (reject overdrafts) and getBalance(). Simulate 5 mixed operations and print the balance after each.

Code:

```
static class Task13 {
    static class BankAccount {
        private double balance;
        BankAccount(double openingBalance) { balance = openingBalance; }
        void deposit(double amount) {
            if (amount <= 0) {
                System.out.println("Deposit " + amount + " rejected (invalid amount)");
            } else {
                balance = balance + amount;
                System.out.println("Deposited " + amount);
            }
            System.out.println("   Balance: " + balance);
        }
        void withdraw(double amount) {
            if (amount <= 0) {
                System.out.println("Withdraw " + amount + " rejected (invalid amount)");
            } else if (amount > balance) {
                System.out.println("Withdraw " + amount + " rejected (not enough
money)");
            } else {
                balance = balance - amount;
                System.out.println("Withdrew " + amount);
            }
            System.out.println("   Balance: " + balance);
        }
        double getBalance() { return balance; }
    }
    static void run() {
        BankAccount account = new BankAccount(500);
        System.out.println("Opening balance: " + account.getBalance());
        account.deposit(200);
        account.withdraw(300);
        account.withdraw(1000);
        account.deposit(-50);
        account.withdraw(400);
    }
}
```

Output:

```
Opening balance: 500.0
Deposited 200.0
   Balance: 700.0
Withdrew 300.0
   Balance: 400.0
Withdraw 1000.0 rejected (not enough money)
   Balance: 400.0
Deposit -50.0 rejected (invalid amount)
   Balance: 400.0
Withdrew 400.0
   Balance: 0.0
```

Task 14 [Non-OOP]

Find the second largest value in an int array of 10 elements without sorting. The array may contain duplicates of the largest value.

Code:

```
static void task14() {
    int[] a = {12, 35, 1, 10, 34, 1, 35, 7, 35, 20};
    System.out.print("Array: ");
    showArray(a);
    int largest = a[0];
    boolean foundSecond = false;
    int second = 0;
    for (int i = 1; i < a.length; i++) {
        if (a[i] > largest) {
            second = largest;
            foundSecond = true;
            largest = a[i];
        } else if (a[i] < largest && (!foundSecond || a[i] > second)) {
            second = a[i];
            foundSecond = true;
        }
    }
    if (foundSecond) {
        System.out.println("Largest = " + largest + ", second largest = " + second);
    } else {
        System.out.println("There is no second largest (all values are equal)");
    }
}
```

Output:

```
Array: [12, 35, 1, 10, 34, 1, 35, 7, 35, 20]
Largest = 35, second largest = 34
```

Task 15 [Non-OOP]

Rotate an array to the left by k positions in place, without creating a second array (k may be larger than the array length).

Code:

```
static void task15() {
    int[] a = {1, 2, 3, 4, 5, 6, 7};
    int k = readInt("Rotate left by k = ");
    System.out.print("Before: ");
    showArray(a);
    int n = a.length;
    k = k % n;
    if (k < 0) k = k + n;
    for (int step = 0; step < k; step++) {
        int first = a[0];
        for (int i = 0; i < n - 1; i++) {
            a[i] = a[i + 1];
        }
        a[n - 1] = first;
```

```
    }
    System.out.print("After:  ");
    showArray(a);
}
```

Output:

```
Before: [1, 2, 3, 4, 5, 6, 7]
After:  [4, 5, 6, 7, 1, 2, 3]
```

```
Before: [1, 2, 3, 4, 5, 6, 7]
After:  [1, 2, 3, 4, 5, 6, 7]
```

Task 16 [Non-OOP]

Print a hollow diamond of stars for an odd n given by the user.

Code:

```
static void task16() {
    int n = readInt("Enter an odd number: ");
    if (n < 1 || n / 2 * 2 == n) {
        System.out.println("n must be a positive odd number");
        return;
    }
    int middle = n / 2;
    for (int row = 0; row < n; row++) {
        int outer = Math.abs(middle - row);
        int inner = n - 2 * outer - 2;
        for (int i = 0; i < outer; i++) System.out.print(" ");
        System.out.print("*");
        if (inner >= 0) {
            for (int i = 0; i < inner; i++) System.out.print(" ");
            System.out.print("*");
        }
        System.out.println();
    }
}
```

Output:

```
Enter an odd number: 7
   *
  * *
 *   *
*     *
 *   *
  * *
   *
```

Task 17 [OOP]

Create a Student class with a name and an array of marks, and methods average() and grade(). Store 3 students in an array and print the topper.

Code:

```
static class Task17 {
    static class Student {
        private String name;
        private int[] marks;
        Student(String name, int[] marks) {
            this.name = name;
            this.marks = marks;
        }
        String getName() { return name; }
        double average() {
            if (marks.length == 0) return 0;
            int sum = 0;
            for (int i = 0; i < marks.length; i++) sum = sum + marks[i];
            return (double) sum / marks.length;
        }
        String grade() {
            double avg = average();
            if (avg >= 90) return "A";
            if (avg >= 80) return "B";
            if (avg >= 70) return "C";
            if (avg >= 60) return "D";
            return "F";
        }
    }
    static void run() {
        Student[] students = new Student[3];
        students[0] = new Student("Ali", new int[]{80, 90, 70});
        students[1] = new Student("Sara", new int[]{95, 92, 98});
        students[2] = new Student("Omar", new int[]{60, 55, 65});
        Student topper = students[0];
        for (int i = 0; i < students.length; i++) {
            Student s = students[i];
            System.out.printf("%s: average = %.2f, grade = %s%n", s.getName(),
s.average(), s.grade());
            if (s.average() > topper.average()) topper = s;
        }
        System.out.println("Topper: " + topper.getName());
    }
}
```

Output:

```
Ali: average = 80.00, grade = B
Sara: average = 95.00, grade = A
Omar: average = 60.00, grade = D
Topper: Sara
```

Task 18 [Non-OOP]

Check whether two Strings are anagrams without using any sorting method.

Code:

```
static boolean areAnagrams(String a, String b) {
    a = a.toLowerCase();
    b = b.toLowerCase();
    int[] count = new int[26];
    for (int i = 0; i < a.length(); i++) {
        char ch = a.charAt(i);
        if (ch >= 'a' && ch <= 'z') count[ch - 'a']++;
    }
    for (int i = 0; i < b.length(); i++) {
        char ch = b.charAt(i);
        if (ch >= 'a' && ch <= 'z') count[ch - 'a']--;
    }
    for (int i = 0; i < 26; i++) {
        if (count[i] != 0) return false;
    }
    return true;
}
static void task18() {
    String a = readText("First string: ");
    String b = readText("Second string: ");
    if (areAnagrams(a, b)) {
        System.out.println("They are anagrams");
    } else {
        System.out.println("They are NOT anagrams");
    }
}
```

Output:

```
First string: listen
Second string: silent
They are anagrams
```

```
First string: hello
Second string: olleh
They are anagrams
```

Task 19 [Non-OOP]

Remove duplicates from an int array while preserving the original order. Print the result without using any Collection class.

Code:

```
static void task19() {
    int[] a = {4, 5, 4, 2, 5, 9, 2, 1, 9};
    System.out.print("Original: ");
    showArray(a);
    int[] result = new int[a.length];
    int size = 0;
    for (int i = 0; i < a.length; i++) {
```

```
        boolean alreadyThere = false;
        for (int j = 0; j < size; j++) {
            if (result[j] == a[i]) alreadyThere = true;
        }
        if (!alreadyThere) {
            result[size] = a[i];
            size++;
        }
    }
    System.out.print("Result:   ");
    for (int i = 0; i < size; i++) System.out.print(result[i] + " ");
    System.out.println();
}
```

Output:

```
Original: [4, 5, 4, 2, 5, 9, 2, 1, 9]
Result:   4 5 2 9 1
```

Task 20 [OOP]

Create an abstract class Shape with an abstract method area(). Extend it with Circle and Triangle. Store mixed objects in a Shape array and print the total area using polymorphism.

Code:

```
static class Task20 {
    static abstract class Shape {
        abstract double area();
    }
    static class Circle extends Shape {
        private double radius;
        Circle(double radius) { this.radius = radius; }
        double area() { return Math.PI * radius * radius; }
    }
    static class Triangle extends Shape {
        private double base, height;
        Triangle(double base, double height) { this.base = base; this.height = height; }
        double area() { return 0.5 * base * height; }
    }
    static void run() {
        Shape[] shapes = new Shape[4];
        shapes[0] = new Circle(2);
        shapes[1] = new Triangle(3, 4);
        shapes[2] = new Circle(1.5);
        shapes[3] = new Triangle(6, 2.5);
        double total = 0;
        for (int i = 0; i < shapes.length; i++) {
            System.out.printf("Shape %d area = %.2f%n", i + 1, shapes[i].area());
            total = total + shapes[i].area();
        }
        System.out.printf("Total area = %.2f%n", total);
    }
}
```

Output:

```
Shape 1 area = 12.57
Shape 2 area = 6.00
Shape 3 area = 7.07
Shape 4 area = 7.50
Total area = 33.13
```

Task 21 [Non-OOP]

Write a recursive method that computes x^n using fast exponentiation. Also count and print how many multiplications were performed for n = 30.

Code:

```
static int multiplications = 0;
static double power(double x, int n) {
    if (n == 0) return 1;
    double half = power(x, n / 2);
    if (n % 2 == 0) {
        multiplications = multiplications + 1;
        return half * half;
    } else {
        multiplications = multiplications + 2;
        return half * half * x;
    }
}
static void task21() {
    double x = readDouble("Enter x: ");
    int n = readInt("Enter n (0 or more): ");
    if (n < 0) {
        System.out.println("Please use n >= 0");
        return;
    }
    multiplications = 0;
    System.out.println(x + "^" + n + " = " + power(x, n));
    multiplications = 0;
    double r = power(x, 30);
    System.out.println(x + "^30 = " + r);
    System.out.println("Multiplications used for n = 30: " + multiplications);
}
```

Output:

```
Enter x: 2
Enter n (0 or more): 30
2.0^30 = 1.073741824E9
2.0^30 = 1.073741824E9
Multiplications used for n = 30: 9
```

Task 22 [Non-OOP]

Read a 3x3 matrix, print its transpose, and report whether the matrix is symmetric.

Code:

```
static void task22() {
    int[][] m = new int[3][3];
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            m[i][j] = readInt("Enter element [" + i + "][" + j + "]: ");
        }
    }
    System.out.println("Matrix:");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) System.out.print(m[i][j] + "\t");
        System.out.println();
    }
    System.out.println("Transpose:");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) System.out.print(m[j][i] + "\t");
        System.out.println();
    }
    boolean symmetric = true;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (m[i][j] != m[j][i]) symmetric = false;
        }
    }
    if (symmetric) {
        System.out.println("The matrix is symmetric");
    } else {
        System.out.println("The matrix is NOT symmetric");
    }
}
```

Output:

```
Enter element [0][0]: 0
Enter element [0][1]: 1
Enter element [0][2]: 0
Enter element [1][0]: 1
Enter element [1][1]: 0
Enter element [1][2]: 1
Enter element [2][0]: 0
Enter element [2][1]: 1
Enter element [2][2]: 0
Matrix:
0	1	0
1	0	1
0	1	0
Transpose:
0	1	0
1	0	1
0	1	0
The matrix is symmetric
```

Task 23 [Non-OOP]

Convert a decimal number to binary without using Integer.toBinaryString(). Solve it once with a loop and once with recursion.

Code:

```
static String binaryWithLoop(int n) {
    if (n == 0) return "0";
    String result = "";
    while (n > 0) {
        result = (n % 2) + result;
        n = n / 2;
    }
    return result;
}
static String binaryWithRecursion(int n) {
    if (n < 2) return "" + n;
    return binaryWithRecursion(n / 2) + (n % 2);
}
static void task23() {
    int n = readInt("Enter a decimal number: ");
    String sign = "";
    if (n < 0) {
        sign = "-";
        n = -n;
    }
    System.out.println("Loop:      " + sign + binaryWithLoop(n));
    System.out.println("Recursion: " + sign + binaryWithRecursion(n));
}
```

Output:

```
Enter a decimal number: 23
Loop:      10111
Recursion: 10111
```

Task 24 [OOP]

Create a Point class with a method distanceTo(Point p). Read three points and decide whether they form a right-angled triangle.

Code:

```
static class Task24 {
    static class Point {
        private double x, y;
        Point(double x, double y) { this.x = x; this.y = y; }
        double getX() { return x; }
        double getY() { return y; }
        double distanceTo(Point p) {
            double dx = x - p.x;
            double dy = y - p.y;
            return Math.sqrt(dx * dx + dy * dy);
        }
    }
    static void run() {
        Point[] p = new Point[3];
```

```
        for (int i = 0; i < 3; i++) {
            double x = readDouble("Point " + (i + 1) + " x: ");
            double y = readDouble("Point " + (i + 1) + " y: ");
            p[i] = new Point(x, y);
        }
        double cross = (p[1].getX() - p[0].getX()) * (p[2].getY() - p[0].getY())
                - (p[2].getX() - p[0].getX()) * (p[1].getY() - p[0].getY());
        if (Math.abs(cross) < 0.000001) {
            System.out.println("These points do not form a triangle");
            return;
        }
        double a = p[0].distanceTo(p[1]);
        double b = p[1].distanceTo(p[2]);
        double c = p[2].distanceTo(p[0]);
        boolean right = Math.abs(a * a + b * b - c * c) < 0.000001
                || Math.abs(b * b + c * c - a * a) < 0.000001
                || Math.abs(a * a + c * c - b * b) < 0.000001;
        if (right) {
            System.out.println("It IS a right-angled triangle");
        } else {
            System.out.println("It is NOT a right-angled triangle");
        }
    }
}
```

Output:

```
Point 1 x: 3
Point 1 y: 4
Point 2 x: 5
Point 2 y: 3
Point 3 x: 4
Point 3 y: 5
It is NOT a right-angled triangle
```

```
Point 1 x: 0
Point 1 y: 0
Point 2 x: 3
Point 2 y: 0
Point 3 x: 0
Point 3 y: 4
It IS a right-angled triangle
```

Task 25 [Non-OOP]

Print the first non-repeating character of a String, or "none" if every character repeats.

Code:

```
static void task25() {
    String text = readText("Enter a string: ");
    for (int i = 0; i < text.length(); i++) {
```

```
        boolean repeats = false;
        for (int j = 0; j < text.length(); j++) {
            if (i != j && text.charAt(i) == text.charAt(j)) repeats = true;
        }
        if (!repeats) {
            System.out.println("First non-repeating character: " + text.charAt(i));
            return;
        }
    }
    System.out.println("none");
}
```

Output:

```
Enter a string: Print the first non-repeating character of a String, or "none"
First non-repeating character: P
```

Task 26 [Non-OOP]

Print all Armstrong numbers between 1 and 1000.

Code:

```
static int intPower(int base, int exponent) {
    int result = 1;
    for (int i = 0; i < exponent; i++) result = result * base;
    return result;
}
static void task26() {
    System.out.println("Armstrong numbers from 1 to 1000:");
    for (int number = 1; number <= 1000; number++) {
        int digitCount = 0;
        int temp = number;
        while (temp > 0) {
            digitCount++;
            temp = temp / 10;
        }
        int sum = 0;
        temp = number;
        while (temp > 0) {
            sum = sum + intPower(temp % 10, digitCount);
            temp = temp / 10;
        }
        if (sum == number) System.out.print(number + " ");
    }
    System.out.println();
}
```

Output:

```
Armstrong numbers from 1 to 1000:
1 2 3 4 5 6 7 8 9 153 370 371 407
```

Task 27 [OOP]

Create a Calculator class with overloaded add() methods for (int,int), (double,double), (int,int,int) and (String,String). Each method must print which version was called. Call them with mixed arguments such as add(5, 2.5), add('a', 1) and add("5", "2").

Code:

```
static class Task27 {
    static class Calculator {
        int add(int a, int b) {
            System.out.println("  add(int, int) was called");
            return a + b;
        }
        double add(double a, double b) {
            System.out.println("  add(double, double) was called");
            return a + b;
        }
        int add(int a, int b, int c) {
            System.out.println("  add(int, int, int) was called");
            return a + b + c;
        }
        String add(String a, String b) {
            System.out.println("  add(String, String) was called");
            return a + b;
        }
    }
    static void run() {
        Calculator calc = new Calculator();
        System.out.println("add(5, 2.5)     -> " + calc.add(5, 2.5));
        System.out.println("add('a', 1)     -> " + calc.add('a', 1));
        System.out.println("add(\"5\", \"2\")   -> " + calc.add("5", "2"));
        System.out.println("add(1, 2, 3)    -> " + calc.add(1, 2, 3));
        System.out.println("add(3, 4)       -> " + calc.add(3, 4));
    }
}
```

Output:

```
  add(double, double) was called
add(5, 2.5)     -> 7.5
  add(int, int) was called
add('a', 1)     -> 98
  add(String, String) was called
add("5", "2")   -> 52
  add(int, int, int) was called
add(1, 2, 3)    -> 6
  add(int, int) was called
add(3, 4)       -> 7
```

Task 28 [Non-OOP]

Compress a String so that "aaabbc" becomes "a3b2c1". Also write the reverse (decompress) method.

Code:

```
static String compress(String s) {
    if (s.length() == 0) return "";
    String result = "";
    char current = s.charAt(0);
    int count = 1;
    for (int i = 1; i < s.length(); i++) {
        if (s.charAt(i) == current) {
            count++;
        } else {
            result = result + current + count;
            current = s.charAt(i);
            count = 1;
        }
    }
    result = result + current + count;
    return result;
}
static String decompress(String s) {
    String result = "";
    int i = 0;
    while (i < s.length()) {
        char letter = s.charAt(i);
        i++;
        int count = 0;
        while (i < s.length() && Character.isDigit(s.charAt(i))) {
            count = count * 10 + (s.charAt(i) - '0');
            i++;
        }
        for (int k = 0; k < count; k++) result = result + letter;
    }
    return result;
}
static void task28() {
    String text = readText("Enter a string (letters only, e.g. aaabbc): ");
    String packed = compress(text);
    System.out.println("Compressed:   " + packed);
    System.out.println("Decompressed: " + decompress(packed));
}
```

Output:

```
Enter a string (letters only, e.g. aaabbc): bbbccdekkkfhhh
Compressed:   b3c2d1e1k3f1h3
Decompressed: bbbccdekkkfhhh
```

Task 29 [OOP]

Create an Employee class and a Manager subclass that overrides calculateSalary() and uses super. Store both types in one Employee array and print every salary.

Code:

```
static class Task29 {
    static class Employee {
        protected String name;
        protected double baseSalary;
```

```
    Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    double calculateSalary() { return baseSalary; }
    String getName() { return name; }
}
static class Manager extends Employee {
    private double bonus;
    Manager(String name, double baseSalary, double bonus) {
        super(name, baseSalary);
        this.bonus = bonus;
    }
    double calculateSalary() {
        return super.calculateSalary() + bonus;
    }
}
static void run() {
    Employee[] staff = new Employee[3];
    staff[0] = new Employee("Ali", 3000);
    staff[1] = new Manager("Sara", 5000, 1500);
    staff[2] = new Employee("Omar", 3500);
    for (int i = 0; i < staff.length; i++) {
        System.out.println(staff[i].getName() + " earns " + staff[i].calculateSalary());
    }
}
}
```

Output:

```
Ali earns 3000.0
Sara earns 6500.0
Omar earns 3500.0
```

Task 30 [Non-OOP]

An array holds the numbers 1 to n with exactly one missing. Find the missing number without sorting and without using an extra array.

Code:

```
static void task30() {
    int[] a = {1, 2, 4, 5, 6, 3, 8, 9, 10};
    int n = a.length + 1;
    System.out.print("Array: ");
    showArray(a);
    int expectedSum = n * (n + 1) / 2;
    int actualSum = 0;
    for (int i = 0; i < a.length; i++) actualSum = actualSum + a[i];
    System.out.println("Missing number: " + (expectedSum - actualSum));
}
```

Output:

```
Array: [1, 2, 4, 5, 6, 3, 8, 9, 10]
Missing number: 7
```

Level 3: Advanced

Task 31 [Non-OOP]

Print Pascal's triangle for n rows using a jagged 2D array.

Code:

```
static void task31() {
    int n = readInt("How many rows? ");
    int[][] triangle = new int[n][];
    for (int i = 0; i < n; i++) {
        triangle[i] = new int[i + 1];
        triangle[i][0] = 1;
        triangle[i][i] = 1;
        for (int j = 1; j < i; j++) {
            triangle[i][j] = triangle[i - 1][j - 1] + triangle[i - 1][j];
        }
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= i; j++) System.out.print(triangle[i][j] + " ");
        System.out.println();
    }
}
```

Output:

```
How many rows? 5
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

Task 32 [OOP]

Build a Stack class using an array with push, pop, peek and isEmpty, handling overflow and underflow. Use it to check whether a String of (), {} and [] brackets is balanced.

Code:

```
static class Task32 {
    static class Stack {
        private char[] data;
        private int top = 0;
        Stack(int size) { data = new char[size]; }
        boolean isEmpty() { return top == 0; }
        void push(char c) {
            if (top == data.length) {
                System.out.println("  Stack overflow: cannot push '" + c + "'");
                return;
            }
            data[top] = c;
            top++;
        }
        char pop() {
```

```
        if (isEmpty()) {
            System.out.println("  Stack underflow: nothing to pop");
            return '?';
        }
        top--;
        return data[top];
    }
    char peek() {
        if (isEmpty()) {
            System.out.println("  Stack is empty: nothing to peek");
            return '?';
        }
        return data[top - 1];
    }
}
static boolean isBalanced(String text) {
    Stack stack = new Stack(text.length());
    for (int i = 0; i < text.length(); i++) {
        char ch = text.charAt(i);
        if (ch == '(' || ch == '{' || ch == '[') {
            stack.push(ch);
        } else if (ch == ')' || ch == '}' || ch == ']') {
            if (stack.isEmpty()) return false;
            char open = stack.pop();
            if (ch == ')' && open != '(') return false;
            if (ch == '}' && open != '{') return false;
            if (ch == ']' && open != '[') return false;
        }
    }
    return stack.isEmpty();
}
static void run() {
    System.out.println("Overflow / underflow demo (stack of size 2):");
    Stack small = new Stack(2);
    small.push('a');
    small.push('b');
    small.push('c');
    small.pop();
    small.pop();
    small.pop();
    String text = readText("Enter a string of brackets: ");
    if (isBalanced(text)) {
        System.out.println("Balanced");
    } else {
        System.out.println("NOT balanced");
    }
}
}
```

Output:

```
Overflow / underflow demo (stack of size 2):
  Stack overflow: cannot push 'c'
  Stack underflow: nothing to pop
Enter a string of brackets: ([]{}){{()()}}{()[])
NOT balanced
```

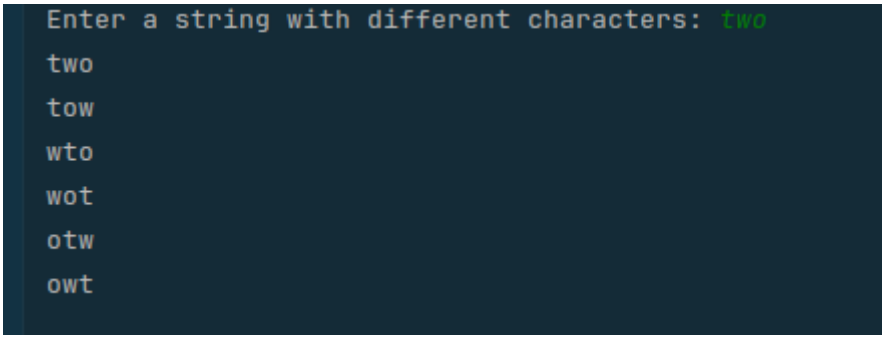
Task 33 [Non-OOP]

Generate and print all permutations of a String with distinct characters using recursion.

Code:

```
static void permute(String done, String left) {
    if (left.length() == 0) {
        System.out.println(done);
        return;
    }
    for (int i = 0; i < left.length(); i++) {
        String remaining = left.substring(0, i) + left.substring(i + 1);
        permute(done + left.charAt(i), remaining);
    }
}
static void task33() {
    String text = readText("Enter a string with different characters: ");
    permute("", text);
}
```

Output:



Task 34 [OOP]

Create class A and class B extends A. Each must have a static block, an instance block and a constructor that print a message. Run new B() twice and print the complete output.

Code:

```
static class Task34 {
    static class A {
        static { System.out.println("A: static block"); }
        { System.out.println("A: instance block"); }
        A() { System.out.println("A: constructor"); }
    }
    static class B extends A {
        static { System.out.println("B: static block"); }
        { System.out.println("B: instance block"); }
        B() { System.out.println("B: constructor"); }
    }
    static void run() {
        System.out.println("--- first new B() ---");
        new B();
        System.out.println("--- second new B() ---");
        new B();
        System.out.println("(static blocks run only once, the first time the class is used)");
    }
}
```

Output:

```
--- first new B() ---
A: static block
B: static block
A: instance block
A: constructor
B: instance block
B: constructor
--- second new B() ---
A: instance block
A: constructor
B: instance block
B: constructor
(static blocks run only once, the first time the class is used)
```

Task 35 **[Non-OOP]**

Write an iterative binary search that also returns the number of steps taken. Then modify it to return the first occurrence in an array containing duplicates.

Code:

```
static int steps = 0;
static int binarySearch(int[] a, int key) {
    steps = 0;
    int low = 0, high = a.length - 1;
    while (low <= high) {
        steps++;
        int mid = (low + high) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
static int firstOccurrence(int[] a, int key) {
    steps = 0;
    int low = 0, high = a.length - 1;
    int answer = -1;
    while (low <= high) {
        steps++;
        int mid = (low + high) / 2;
        if (a[mid] == key) {
            answer = mid;
            high = mid - 1;
        } else if (a[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
```

```
    return answer;
}
static void task35() {
    int[] sorted = {1, 3, 5, 7, 9, 11, 13, 15};
    int[] withDuplicates = {2, 4, 4, 4, 4, 7, 9, 9};
    int key = readInt("Number to search for: ");
    System.out.print("Array 1: ");
    showArray(sorted);
    int index = binarySearch(sorted, key);
    System.out.println("binarySearch -> index " + index + " in " + steps + " step(s)");
    System.out.print("Array 2: ");
    showArray(withDuplicates);
    index = firstOccurrence(withDuplicates, key);
    System.out.println("firstOccurrence -> index " + index + " in " + steps + " step(s)");
}
```

Output:

```
Number to search for: 7
Array 1: [1, 3, 5, 7, 9, 11, 13, 15]
binarySearch -> index 3 in 1 step(s)
Array 2: [2, 4, 4, 4, 4, 7, 9, 9]
firstOccurrence -> index 5 in 3 step(s)
```

Task 36 [Non-OOP]

Find the longest substring without repeating characters in a given String. Print the substring and its length.

Code:

```
static void task36() {
    String s = readText("Enter a string: ");
    String best = "";
    for (int start = 0; start < s.length(); start++) {
        for (int end = start; end < s.length(); end++) {
            boolean repeated = false;
            for (int k = start; k < end; k++) {
                if (s.charAt(k) == s.charAt(end)) repeated = true;
            }
            if (repeated) break;
            if (end - start + 1 > best.length()) {
                best = s.substring(start, end + 1);
            }
        }
    }
    System.out.println("Longest substring: \"" + best + "\"");
    System.out.println("Length: " + best.length());
}
```

Output:

```
Enter a string: helloimquinn
Longest substring: "loimqu"
Length: 6
```

Task 37 [OOP]

Create an immutable class Money (final fields, no setters) with add(Money) returning a new object. Override equals() and hashCode(), and demonstrate the difference between == and equals().

Code:

```
static class Task37 {
   static final class Money {
      private final int amount;
      private final String currency;
      Money(int amount, String currency) {
         this.amount = amount;
         this.currency = currency;
      }
      Money add(Money other) {
         // returns a NEW object; this object never changes
         return new Money(this.amount + other.amount, currency);
      }
      public boolean equals(Object o) {
         if (this == o) return true;
         if (!(o instanceof Money)) return false;
         Money other = (Money) o;
         return amount == other.amount && currency.equals(other.currency);
      }
      public int hashCode() {
         return 31 * amount + currency.hashCode();
      }
      public String toString() { return amount + " " + currency; }
   }
   static void run() {
      Money m1 = new Money(100, "USD");
      Money m2 = new Money(100, "USD");
      Money m3 = m1;
      System.out.println("m1 == m2     : " + (m1 == m2));
      System.out.println("m1.equals(m2) : " + m1.equals(m2));
      System.out.println("m1 == m3     : " + (m1 == m3));
      System.out.println("same hashCode : " + (m1.hashCode() == m2.hashCode()));
      Money sum = m1.add(m2);
      System.out.println("m1 + m2 = " + sum + "   (m1 is still " + m1 + ")");
   }
}
```

Output:

```
m1 == m2       : false
m1.equals(m2) : true
m1 == m3       : true
same hashCode : true
m1 + m2 = 200 USD   (m1 is still 100 USD)
```

Task 38 [Non-OOP]

Print an n x m matrix in spiral order.

Code:

```
static void task38() {
    int rows = readInt("Rows (n): ");
    int cols = readInt("Columns (m): ");
    int[][] m = new int[rows][cols];
    int value = 1;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            m[i][j] = value;
            value++;
        }
    }
    System.out.println("Matrix:");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) System.out.print(m[i][j] + "\t");
        System.out.println();
    }
    int top = 0, bottom = rows - 1, left = 0, right = cols - 1;
    System.out.print("Spiral: ");
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) System.out.print(m[top][j] + " ");
        top++;
        for (int i = top; i <= bottom; i++) System.out.print(m[i][right] + " ");
        right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) System.out.print(m[bottom][j] + " ");
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) System.out.print(m[i][left] + " ");
            left++;
        }
    }
    System.out.println();
}
```

Output:

```
Rows (n): 3
Columns (m): 4
Matrix:
1   2   3   4
5   6   7   8
9   10  11  12
Spiral: 1 2 3 4 8 12 11 10 9 5 6 7
```

Task 39 [OOP]

Implement a singly linked list from scratch (Node class plus LinkedList class) with insertAtEnd, deleteByValue and an iterative reverse. Print the list before and after reversing.

Code:

```
static class Task39 {
    static class Node {
        int value;
        Node next;
        Node(int value) { this.value = value; }
    }
    static class LinkedList {
        private Node head;
        void insertAtEnd(int value) {
            Node newNode = new Node(value);
            if (head == null) {
                head = newNode;
                return;
            }
            Node current = head;
            while (current.next != null) current = current.next;
            current.next = newNode;
        }
        boolean deleteByValue(int value) {
            if (head == null) return false;
            if (head.value == value) {
                head = head.next;
                return true;
            }
            Node current = head;
            while (current.next != null) {
                if (current.next.value == value) {
                    current.next = current.next.next;
                    return true;
                }
                current = current.next;
            }
            return false;
        }
        void reverse() {
            Node previous = null;
            Node current = head;
            while (current != null) {
                Node next = current.next;
                current.next = previous;
                previous = current;
                current = next;
            }
            head = previous;
        }
        void print() {
            Node current = head;
            while (current != null) {
                System.out.print(current.value);
                if (current.next != null) System.out.print(" -> ");
                current = current.next;
            }
            System.out.println();
        }
    }
    static void run() {
        LinkedList list = new LinkedList();
        for (int v = 10; v <= 50; v += 10) list.insertAtEnd(v);
        System.out.print("List:            ");
```

```
        list.print();
        list.deleteByValue(30);
        System.out.print("After deleting 30: ");
        list.print();
        if (!list.deleteByValue(99)) System.out.println("99 is not in the list");
        System.out.print("Before reverse:   ");
        list.print();
        list.reverse();
        System.out.print("After reverse:    ");
        list.print();
    }
}
```

Output:

```
List:              10 -> 20 -> 30 -> 40 -> 50
After deleting 30: 10 -> 20 -> 40 -> 50
99 is not in the list
Before reverse:    10 -> 20 -> 40 -> 50
After reverse:     50 -> 40 -> 20 -> 10
```

Task 40 [Non-OOP]

Solve Tower of Hanoi for n disks. Print every move and verify that the total equals 2^n - 1.

Code:

```
static int moves = 0;
static void hanoi(int n, char from, char to, char helper) {
    if (n == 0) return;
    hanoi(n - 1, from, helper, to);
    moves++;
    System.out.println("Move disk " + n + " from " + from + " to " + to);
    hanoi(n - 1, helper, to, from);
}
static void task40() {
    int n = readInt("Number of disks (small, e.g. 3): ");
    moves = 0;
    hanoi(n, 'A', 'C', 'B');
    int expected = 1;
    for (int i = 0; i < n; i++) expected = expected * 2;
    expected = expected - 1;
    System.out.println("Total moves = " + moves + ", 2^n - 1 = " + expected);
    if (moves == expected) System.out.println("Verified: they are equal");
    else System.out.println("Something is wrong: they are different");
}
```

Output:

```
Number of disks (small, e.g. 3): 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves = 7, 2^n - 1 = 7
Verified: they are equal
```

Task 41 [OOP]

Create a checked custom exception InsufficientFundsException and use it in a withdraw() method. Show a case where a finally block runs even though the try block executes a return statement.

Code:

```
static class Task41 {
    static class InsufficientFundsException extends Exception {
        InsufficientFundsException(String message) { super(message); }
    }
    static class Account {
        private double balance;
        Account(double balance) { this.balance = balance; }
        void withdraw(double amount) throws InsufficientFundsException {
            if (amount > balance) {
                throw new InsufficientFundsException(
                        "Cannot withdraw " + amount + ", balance is only " + balance);
            }
            balance = balance - amount;
        }
        double getBalance() { return balance; }
    }
    static String finallyDemo() {
        try {
            System.out.println("  inside try: about to return");
            return "value returned from try";
        } finally {
            System.out.println("  inside finally: still runs!");
        }
    }
    static void run() {
        Account account = new Account(100);
        try {
            account.withdraw(30);
            System.out.println("Withdrew 30, balance = " + account.getBalance());
            account.withdraw(500);
            System.out.println("This line is never reached");
        } catch (InsufficientFundsException e) {
            System.out.println("Caught: " + e.getMessage());
```

```
        }
        System.out.println("finally demo:");
        System.out.println("  result = " + finallyDemo());
    }
}
```

Output:

```
Withdrew 30, balance = 70.0
Caught: Cannot withdraw 500.0, balance is only 70.0
finally demo:
  inside try: about to return
  inside finally: still runs!
  result = value returned from try
```

Task 42 [Non-OOP]

Merge two sorted int arrays into one sorted array in O(n + m) time without calling any sort method.

Code:

```
static void task42() {
    int[] a = {1, 4, 7, 10, 15};
    int[] b = {2, 3, 8, 9, 20, 25};
    int[] merged = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) {
        if (a[i] <= b[j]) {
            merged[k] = a[i];
            i++;
        } else {
            merged[k] = b[j];
            j++;
        }
        k++;
    }
    while (i < a.length) { merged[k] = a[i]; i++; k++; }
    while (j < b.length) { merged[k] = b[j]; j++; k++; }
    System.out.print("a:      "); showArray(a);
    System.out.print("b:      "); showArray(b);
    System.out.print("merged: "); showArray(merged);
}
```

Output:

```
a:      [1, 4, 7, 10, 15]
b:      [2, 3, 8, 9, 20, 25]
merged: [1, 2, 3, 4, 7, 8, 9, 10, 15, 20, 25]
```

Level 4: Expert (Critical Thinking)

Task 43 [OOP]

Implement a Singleton class with lazy initialization. Prove that only one instance exists by calling it 5 times, then make it thread-safe.

Code:

```
static class Task43 {
    // Version 1: lazy, but NOT safe when several threads call it at the same time
    static class LazySingleton {
        private static LazySingleton instance;

        private LazySingleton() {
            System.out.println("  (creating the instance)");
        }
        static LazySingleton getInstance() {
            if (instance == null) {
                instance = new LazySingleton();
            }
            return instance;
        }
    }
    // Version 2: "synchronized" lets only one thread in at a time
    static class SafeSingleton {
        private static SafeSingleton instance;

        private SafeSingleton() {
            System.out.println("  (creating the safe instance)");
        }

        static synchronized SafeSingleton getInstance() {
            if (instance == null) {
                instance = new SafeSingleton();
            }
            return instance;
        }
    }
    static class Worker extends Thread {
        private SafeSingleton first;
        Worker(SafeSingleton first) { this.first = first; }

        public void run() {
            SafeSingleton s = SafeSingleton.getInstance();
            System.out.println("  thread got the same instance? " + (s == first));
        }
    }
    static void run() {
        System.out.println("Lazy singleton, 5 calls:");
        LazySingleton first = LazySingleton.getInstance();
        for (int i = 1; i <= 5; i++) {
            LazySingleton s = LazySingleton.getInstance();
            System.out.println("  call " + i + ": same instance as first? " + (s == first));
        }
        System.out.println("Thread-safe singleton, 5 threads:");
        SafeSingleton safeFirst = SafeSingleton.getInstance();
        Worker[] workers = new Worker[5];
        for (int i = 0; i < 5; i++) {
```

```
            workers[i] = new Worker(safeFirst);
            workers[i].start();
        }
        for (int i = 0; i < 5; i++) {
            try {
                workers[i].join();
            } catch (InterruptedException e) {
                System.out.println("Interrupted");
            }
        }
    }
}
```

Output:

```
Lazy singleton, 5 calls:
  (creating the instance)
  call 1: same instance as first? true
  call 2: same instance as first? true
  call 3: same instance as first? true
  call 4: same instance as first? true
  call 5: same instance as first? true
Thread-safe singleton, 5 threads:
  (creating the safe instance)
  thread got the same instance? true
  thread got the same instance? true
  thread got the same instance? true
  thread got the same instance? true
  thread got the same instance? true
```

Task 44 [Non-OOP]

Solve the N-Queens problem for N = 6. Print the total number of solutions and display one valid board.

Code:

```
static int N = 6;
static int[] queenColumn = new int[N];
static int[] firstBoard = null;
static int solutionCount = 0;
static boolean isSafe(int row, int col) {
    for (int r = 0; r < row; r++) {
        if (queenColumn[r] == col) return false;
        if (Math.abs(queenColumn[r] - col) == row - r) return false;
    }
    return true;
}
static void placeQueen(int row) {
    if (row == N) {
        solutionCount++;
        if (solutionCount == 1) {
```

```
            firstBoard = new int[N];
            for (int i = 0; i < N; i++) firstBoard[i] = queenColumn[i];
        }
        return;
    }
    for (int col = 0; col < N; col++) {
        if (isSafe(row, col)) {
            queenColumn[row] = col;
            placeQueen(row + 1);
        }
    }
}
static void task44() {
    solutionCount = 0;
    firstBoard = null;
    placeQueen(0);
    System.out.println("Total solutions for N = " + N + ": " + solutionCount);
    System.out.println("One valid board:");
    for (int row = 0; row < N; row++) {
        for (int col = 0; col < N; col++) {
            if (firstBoard[row] == col) System.out.print("Q ");
            else System.out.print(". ");
        }
        System.out.println();
    }
}
```

Output:

```
Total solutions for N = 6: 4
One valid board:
. Q . . . .
. . . Q . .
. . . . . Q
Q . . . . .
. . Q . . .
. . . . Q .
```

Task 45 [OOP]

Create two interfaces A and B that both have a default method hello(). Write a class C that implements both, resolves the conflict, and calls each interface's version.

Code:

```
static class Task45 {
    interface A {
        default void hello() { System.out.println("Hello from A"); }
    }
    interface B {
        default void hello() { System.out.println("Hello from B"); }
    }
    static class C implements A, B {
        public void hello() {
```

```
        A.super.hello();
        B.super.hello();
    }
  }
  static void run() {
    new C().hello();
  }
}
```

Output:

```
Hello from A
Hello from B
```

Task 46 [Non-OOP]

Find the contiguous subarray with the maximum sum (Kadane's algorithm). Print the sum with the start and end indices, and make sure an all-negative array works.

Code:

```
static void kadane(int[] a) {
  int best = a[0];
  int bestStart = 0, bestEnd = 0;
  int current = a[0];
  int currentStart = 0;
  for (int i = 1; i < a.length; i++) {
    if (current < 0) {
      current = a[i];
      currentStart = i;
    } else {
      current = current + a[i];
    }
    if (current > best) {
      best = current;
      bestStart = currentStart;
      bestEnd = i;
    }
  }
  System.out.print("Array: ");
  showArray(a);
  System.out.println("Max sum = " + best + " (from index " + bestStart + " to " +
bestEnd + ")");
}
static void task46() {
  kadane(new int[]{-2, 1, -3, 4, -1, 2, 1, -5, 4});
  kadane(new int[]{-8, -3, -6, -2, -5});
}
```

Output:

```
Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Max sum = 6 (from index 3 to 6)
Array: [-8, -3, -6, -2, -5]
Max sum = -2 (from index 3 to 3)
```

Task 47 [OOP]

Create a generic class Pair<K, V> and a generic method max() that works on any array of Comparable objects. Test it with Integer, String and a custom Student class that implements Comparable.

Code:

```
static class Task47 {
    static class Pair<K, V> {
        private K key;
        private V value;
        Pair(K key, V value) { this.key = key; this.value = value; }
        K getKey() { return key; }
        V getValue() { return value; }
        public String toString() { return "(" + key + ", " + value + ")"; }
    }
    static class Student implements Comparable<Student> {
        private String name;
        private int marks;
        Student(String name, int marks) { this.name = name; this.marks = marks; }
        public int compareTo(Student other) {
            return Integer.compare(marks, other.marks);
        }
        public String toString() { return name + " (" + marks + ")"; }
    }
    static <T extends Comparable<T>> T max(T[] items) {
        T biggest = items[0];
        for (int i = 1; i < items.length; i++) {
            if (items[i].compareTo(biggest) > 0) biggest = items[i];
        }
        return biggest;
    }
    static void run() {
        Pair<String, Integer> pair = new Pair<>("age", 20);
        System.out.println("Pair: " + pair + "  key=" + pair.getKey() + "  value=" +
pair.getValue());
        Integer[] numbers = {3, 9, 4, 7};
        String[] words = {"pear", "apple", "zebra", "mango"};
        Student[] students = {new Student("Ali", 80), new Student("Sara", 95), new
Student("Omar", 60)};
        System.out.println("max of Integers: " + max(numbers));
        System.out.println("max of Strings:  " + max(words));
        System.out.println("max of Students: " + max(students));
    }
}
```

Output:

```
Pair: (age, 20)  key=age  value=20
max of Integers: 9
max of Strings:  zebra
max of Students: Sara (95)
```

Task 48 [Non-OOP]

Write a method that validates a 9x9 Sudoku grid, checking every row, every column and every 3x3 box.

Code:

```
static boolean isValidSudoku(int[][] g) {
    for (int row = 0; row < 9; row++) {
        boolean[] seen = new boolean[10];
        for (int col = 0; col < 9; col++) {
            int v = g[row][col];
            if (v < 1 || v > 9 || seen[v]) return false;
            seen[v] = true;
        }
    }
    for (int col = 0; col < 9; col++) {
        boolean[] seen = new boolean[10];
        for (int row = 0; row < 9; row++) {
            int v = g[row][col];
            if (v < 1 || v > 9 || seen[v]) return false;
            seen[v] = true;
        }
    }
    for (int boxRow = 0; boxRow < 9; boxRow += 3) {
        for (int boxCol = 0; boxCol < 9; boxCol += 3) {
            boolean[] seen = new boolean[10];
            for (int r = 0; r < 3; r++) {
                for (int c = 0; c < 3; c++) {
                    int v = g[boxRow + r][boxCol + c];
                    if (v < 1 || v > 9 || seen[v]) return false;
                    seen[v] = true;
                }
            }
        }
    }
    return true;
}
static void task48() {
    int[][] grid = {
        {5, 3, 4, 6, 7, 8, 9, 1, 2},
        {6, 7, 2, 1, 9, 5, 3, 4, 8},
        {1, 9, 8, 3, 4, 2, 5, 6, 7},
        {8, 5, 9, 7, 6, 1, 4, 2, 3},
        {4, 2, 6, 8, 5, 3, 7, 9, 1},
        {7, 1, 3, 9, 2, 4, 8, 5, 6},
        {9, 6, 1, 5, 3, 7, 2, 8, 4},
        {2, 8, 7, 4, 1, 9, 6, 3, 5},
        {3, 4, 5, 2, 8, 6, 1, 7, 9}
    };
    System.out.println("Correct grid valid?   " + isValidSudoku(grid));
    grid[0][0] = 3;
    System.out.println("Changed grid valid?   " + isValidSudoku(grid));
}
```

Output:

```
Correct grid valid?   true
Changed grid valid?   false
```

Task 49 [OOP]

Build a mini library system with Book, Member and Library classes. A member can borrow at most 3 books, and an already issued book cannot be borrowed again. Simulate borrow and return operations and print the status after each.

Code:

```
static class Task49 {
    static class Book {
        private String title;
        private boolean issued = false;
        Book(String title) { this.title = title; }
        String getTitle() { return title; }
        boolean isIssued() { return issued; }
        void setIssued(boolean issued) { this.issued = issued; }
    }
    static class Member {
        private String name;
        private Book[] borrowed = new Book[3];
        private int count = 0;
        Member(String name) { this.name = name; }
        String getName() { return name; }
        boolean canBorrow() { return count < 3; }
        void addBook(Book b) {
            borrowed[count] = b;
            count++;
        }
        boolean removeBook(Book b) {
            for (int i = 0; i < count; i++) {
                if (borrowed[i] == b) {
                    for (int j = i; j < count - 1; j++) {
                        borrowed[j] = borrowed[j + 1];
                    }
                    borrowed[count - 1] = null;
                    count--;
                    return true;
                }
            }
            return false;
        }
        String holdings() {
            String s = "";
            for (int i = 0; i < count; i++) {
                if (i > 0) s = s + ", ";
                s = s + borrowed[i].getTitle();
            }
            return count == 0 ? "nothing" : s;
        }
    }
    static class Library {
        private Book[] books;
        Library(Book[] books) { this.books = books; }
        void borrow(Member m, Book b) {
            if (b.isIssued()) {
                System.out.println(m.getName() + " cannot borrow '" + b.getTitle() + "': 
already issued");
            } else if (!m.canBorrow()) {
```

```
            System.out.println(m.getName() + " cannot borrow '" + b.getTitle() + "': limit of 3 books reached");
        } else {
            b.setIssued(true);
            m.addBook(b);
            System.out.println(m.getName() + " borrowed '" + b.getTitle() + "'");
        }
        status(m);
    }
    void giveBack(Member m, Book b) {
        if (m.removeBook(b)) {
            b.setIssued(false);
            System.out.println(m.getName() + " returned '" + b.getTitle() + "'");
        } else {
            System.out.println(m.getName() + " does not have '" + b.getTitle() + "'");
        }
        status(m);
    }
    void status(Member m) {
        String available = "";
        for (int i = 0; i < books.length; i++) {
            if (!books[i].isIssued()) {
                if (!available.isEmpty()) available = available + ", ";
                available = available + books[i].getTitle();
            }
        }
        System.out.println("   " + m.getName() + " holds: " + m.holdings());
        System.out.println("   Available: " + (available.isEmpty() ? "none" : available));
    }
}
static void run() {
    Book b1 = new Book("Java Basics");
    Book b2 = new Book("Data Structures");
    Book b3 = new Book("Algorithms");
    Book b4 = new Book("Databases");
    Library library = new Library(new Book[]{b1, b2, b3, b4});
    Member ali = new Member("Ali");
    Member sara = new Member("Sara");
    library.borrow(ali, b1);
    library.borrow(ali, b2);
    library.borrow(sara, b1);
    library.borrow(ali, b3);
    library.borrow(ali, b4);
    library.giveBack(ali, b1);
    library.borrow(sara, b1);
    library.giveBack(sara, b2);
}
}
```

Output:

```
Ali borrowed 'Java Basics'
   Ali holds: Java Basics
   Available: Data Structures, Algorithms, Databases
Ali borrowed 'Data Structures'
   Ali holds: Java Basics, Data Structures
   Available: Algorithms, Databases
Sara cannot borrow 'Java Basics': already issued
   Sara holds: nothing
   Available: Algorithms, Databases
Ali borrowed 'Algorithms'
   Ali holds: Java Basics, Data Structures, Algorithms
   Available: Databases
Ali cannot borrow 'Databases': limit of 3 books reached
   Ali holds: Java Basics, Data Structures, Algorithms
   Available: Databases
Ali returned 'Java Basics'
   Ali holds: Data Structures, Algorithms
   Available: Java Basics, Databases
Sara borrowed 'Java Basics'
   Sara holds: Java Basics
   Available: Databases
Sara does not have 'Data Structures'
   Sara holds: Java Basics
   Available: Databases
```

Task 50 [Non-OOP]

Compute the factorial of 50 without BigInteger by storing the digits in an int array. Print the full result.

Code:

```
static void task50() {
      int[] digits = new int[100];
      digits[0] = 1;
      int length = 1;
      for (int factor = 2; factor <= 50; factor++) {
         int carry = 0;
         for (int i = 0; i < length; i++) {
            int product = digits[i] * factor + carry;
            digits[i] = product % 10;
            carry = product / 10;
         }
         while (carry > 0) {
            digits[length] = carry % 10;
            length++;
```

```
            carry = carry / 10;
        }
    }
    System.out.print("50! = ");
    for (int i = length - 1; i >= 0; i--) System.out.print(digits[i]);
    System.out.println("  (" + length + " digits)");
  }
}
```

Output:

```
50! = 30414093201713378043612608166064768844377641568960512000000000000  (65 digits)
```